

Lab-PM: Bygg ett terminalbaserat chattforum i C via SSH

1. Syfte

Syftet med labben är att själv konstruera ett enkelt terminalbaserat chattforum i C, där flera användare kan ansluta till samma chattrum via en Linux-server.

Labben ska ge träning i:

- nätverksprogrammering med sockets i C
- klient/server-arkitektur
- hantering av flera samtidiga klienter
- trådar med `pthread`
- enkel textbaserad kommunikation över TCP
- kompilering med `gcc`
- felsökning med terminalverktyg
- körning av program på en Linux-server via SSH

Tanken är inte att kopiera färdig kod. Du ska själv skriva implementationen steg för steg utifrån krav, delmål och testfall.

2. Målbild

När labben är klar ska systemet fungera ungefär så här:

```
Användare 1 → SSH till server → startar chattklient → skriver meddelanden
Användare 2 → SSH till server → startar chattklient → ser och skriver
meddelanden
Användare 3 → SSH till server → startar chattklient → deltar i samma chattrum
```

Ett tänkt användarflöde:

```
ssh användare@server-ip
chat
```

Därefter ska användaren kunna välja ett namn och delta i chatten.

3. Avgränsning

Detta är en enkel prototyp, inte ett färdigt chatsystem.

Systemet behöver inte ha:

- grafiskt gränssnitt
- kryptering i själva chattprogrammet
- användarkonton i chatten
- lösenord i chatten
- databas
- flera rum
- moderering
- sparad historik

Systemet ska ha:

- en server skriven i C
- en klient skriven i C
- TCP-kommunikation
- stöd för flera samtidiga klienter
- användarnamn
- möjlighet att lämna med `/quit`
- rimlig felhantering
- instruktioner för kompilering och körning

4. Rekommenderad teknisk lösning

Bygg två separata program:

1. Chattserver

2. Startas på Linux-servern.
3. Lyssnar på en lokal TCP-port.
4. Tar emot klientanslutningar.
5. Håller reda på anslutna klienter.
6. Vidarebefordrar meddelanden till andra klienter.

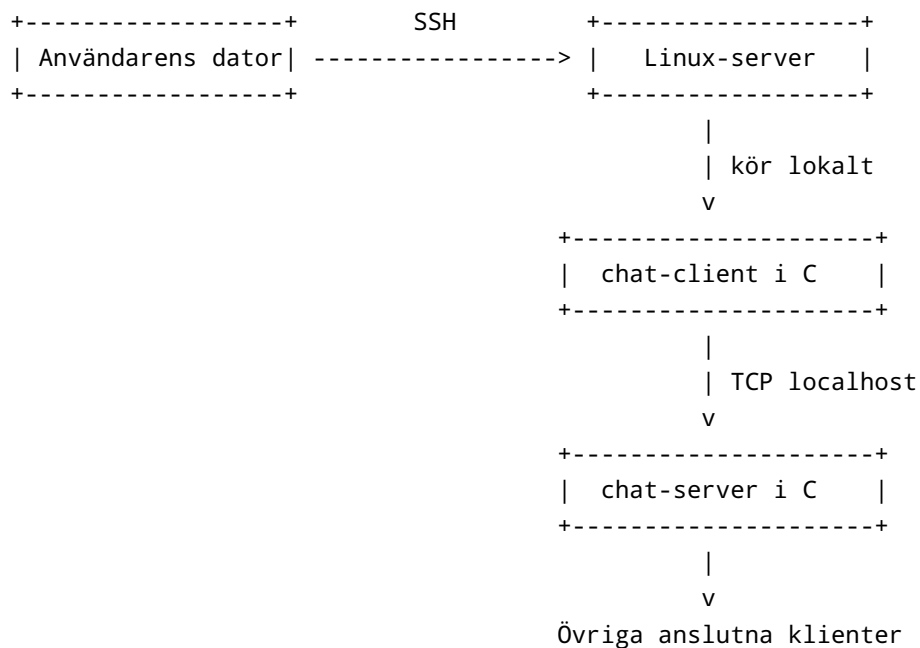
7. Chatklient

8. Körs efter SSH-inloggning.
9. Ansluter till servern via TCP.
10. Läser användarens input från terminalen.
11. Tar emot och visar meddelanden från servern.

Rekommenderad modell:

- TCP sockets
- `pthread` för samtidighet
- en tråd per ansluten klient på serversidan
- en tråd för mottagning på klientsidan

5. Systemskiss



Rekommenderat:

Chattservern lyssnar på 127.0.0.1
SSH är enda vägen in från nätverket

Då behöver du inte öppna chattporten i brandväggen.

6. Förkunskaper

Du bör ha grundläggande koll på:

- C-programmering
- pekare
- arrayer och strängar
- `struct`
- filuppdelning i `.c` och `.h`
- kompilering med `gcc`
- terminal och Linux
- enklare process- och trådbegrepp

Du behöver inte kunna sockets i förväg. Det är en central del av labben.

7. Begrepp att undersöka innan kodning

Innan du börjar skriva programmet ska du kort undersöka och anteckna vad följande betyder:

Sockets

- Vad gör `socket()` ?
- Vad gör `bind()` ?
- Vad gör `listen()` ?
- Vad gör `accept()` ?
- Vad gör `connect()` ?
- Vad gör `send()` ?
- Vad gör `recv()` ?
- Vad betyder en file descriptor?

Adresser och portar

- Vad är `struct sockaddr_in` ?
- Vad betyder `AF_INET` ?
- Vad betyder `SOCK_STREAM` ?
- Vad är skillnaden mellan IP-adress och port?
- Vad betyder `127.0.0.1` ?
- Vad betyder `INADDR_ANY` ?

Trådar

- Vad gör `pthread_create()` ?
- Vad gör `pthread_detach()` ?
- Vad gör `pthread_join()` ?
- Vad är en mutex?
- Varför kan flera trådar skapa problem när de ändrar samma data?

Strängar i C

- Hur fungerar nullterminerade strängar?
- Vad är skillnaden mellan buffertstorlek och faktisk textlängd?
- Vad kan gå fel med `strcpy()` ?
- Varför bör man vara försiktig med användarinput?

8. Föreslagen filstruktur

En möjlig struktur:

```
terminal-chat-c/  
├─ Makefile  
├─ README.md  
├─ notes.md  
├─ src/  
│   └─ server.c
```

```
|   |─ client.c
|   |─ common.h
|   |─ common.c
|   └─ build/
```

Du får förenkla detta i början.

En minimal start kan vara:

```
terminal-chat-c/
├─ server.c
├─ client.c
└─ Makefile
```

9. Kompilering

Du ska kunna kompilera programmen med `gcc`.

Exempel på målbild:

```
make
```

ska skapa två körbara filer, exempelvis:

```
server
client
```

Du ska själv skriva `Makefile`, men den bör kunna:

- kompilera servern
- kompilera klienten
- länka med pthread när det behövs
- rensa byggfiler med `make clean`

Saker att undersöka:

- Vad gör flaggan `-Wall`?
- Vad gör flaggan `-Wextra`?
- Vad gör flaggan `-g`?
- Varför behövs `-pthread`?

Rekommenderade kompileringsflaggor:

```
-Wall -Wextra -g -pthread
```

10. Del 1: Designa chattprotokollet

Innan du skriver kod ska du bestämma hur klient och server ska prata med varandra.

Besvara i `notes.md`:

1. Vad skickar klienten först när den ansluter?
2. Hur väljer användaren namn?
3. Hur ser ett vanligt meddelande ut?
4. Hur lämnar användaren chatten?
5. Ska servern skicka systemmeddelanden, till exempel när någon ansluter?
6. Ska avsändaren själv få se sitt meddelande via servern?
7. Hur ska tomma meddelanden hanteras?
8. Hur långt får ett meddelande vara?
9. Vad händer om användaren skickar mer data än bufferten klarar?

Ett enkelt protokoll kan beskrivas så här:

1. Klienten ansluter.
2. Klienten skickar användarnamn som första rad.
3. Därefter skickar klienten varje textmeddelande som en rad.
4. Om raden är `/quit` stängs anslutningen.
5. Servern skickar inkommande meddelanden till övriga klienter.

Detta är en specifikation, inte kod.

11. Del 2: Bygg en minimal TCP-server

Börja med en server som bara klarar en klient.

Servern ska kunna:

- skapa en socket
- binda socketen till en port
- lyssna efter anslutningar
- acceptera en klient
- ta emot text från klienten
- skriva ut texten i serverns terminal
- stänga anslutningen när klienten kopplar från

Krav

Servern ska inte krascha om klienten stänger anslutningen.

Test

Starta servern i en terminal.

Starta klienten i en annan terminal.

Skicka några meddelanden.

Kontrollera att servern skriver ut dem.

Kontrollfrågor

- Vad returnerar `recv()` när klienten stänger anslutningen?
 - Vad händer om `bind()` misslyckas?
 - Varför kan en port vara upptagen?
 - Vad gör `SO_REUSEADDR`?
-

12. Del 3: Bygg en minimal TCP-klient

Klienten ska kunna:

- skapa en socket
- ansluta till servern
- läsa text från terminalen
- skicka text till servern
- avsluta när användaren skriver `/quit`
- hantera att servern inte är igång

Test

1. Starta servern.
2. Starta klienten.
3. Skriv ett meddelande i klienten.
4. Kontrollera att servern tar emot det.
5. Skriv `/quit`.
6. Kontrollera att anslutningen stängs rent.

Kontrollfrågor

- Vad händer om `connect()` misslyckas?
 - Hur visar du ett begripligt felmeddelande?
 - Vad är skillnaden mellan `send()` och `printf()`?
 - Varför behöver texten skickas som bytes över socketen?
-

13. Del 4: Lägg till flera klienter

Nu ska servern kunna hantera flera samtidiga klienter.

Detta är labbens kärna.

En rimlig lösning i C är:

Huvudtråden:

accepterar nya klienter

Klienttråd:

hanterar en specifik klient

läser meddelanden från klienten

skickar vidare meddelanden till andra klienter

Du ska själv implementera detta med `pthread`.

Krav

- Minst två klienter ska kunna vara anslutna samtidigt.
- Om klient A skriver ska klient B se meddelandet.
- Om klient B skriver ska klient A se meddelandet.
- Om en klient lämnar ska övriga fortsätta fungera.

Viktigt designproblem

Servern behöver hålla reda på alla anslutna klienter.

Fundera på:

- Ska du använda en array av klienter?
- Ska varje klient representeras av en `struct`?
- Vad ska klientstrukturen innehålla?
- Hur markerar du en ledig plats?
- Hur tar du bort en klient som kopplat från?

Exempel på information som kan behövas per klient:

```
socket/file descriptor  
användarnamn  
aktiv/inaktiv status  
tråd-id
```

Detta är inte kod, utan en designskiss.

14. Del 5: Skydda delad data

När flera trådar använder samma klientlista kan problem uppstå.

Exempel:

- en tråd går igenom klientlistan för att skicka meddelanden
- samtidigt tar en annan tråd bort en klient

Det kan ge fel, krascher eller odefinierat beteende.

Därför behöver du använda en mutex runt delad data.

Saker att undersöka

- Vad gör `pthread_mutex_lock()`?
- Vad gör `pthread_mutex_unlock()`?
- Vilken data behöver skyddas?
- När ska man inte hålla låset för länge?

Krav

Klientlistan ska inte ändras oskyddat från flera trådar samtidigt.

15. Del 6: Broadcast av meddelanden

Servern ska kunna skicka ett meddelande från en klient till andra klienter.

Designa en funktion eller logisk del som gör detta.

Den bör kunna:

- ta emot ett färdigt textmeddelande
- gå igenom alla aktiva klienter
- skicka texten till rätt mottagare
- hantera misslyckad sändning

Designval

Ska servern skicka meddelandet till:

1. alla klienter inklusive avsändaren, eller
2. alla klienter utom avsändaren?

Båda fungerar, men beteendet blir olika.

Kontrollfrågor

- Vad händer om `send()` misslyckas?
 - Kan `send()` skicka färre bytes än du bad om?
 - Hur hanterar du klienter som inte längre svarar?
 - Ska ett trasigt klient-socket tas bort direkt?
-

16. Del 7: Användarnamn och systemmeddelanden

När en klient ansluter ska den välja användarnamn.

Servern ska sedan kunna skicka systemmeddelanden som:

```
* mats joined the chat *  
* anna left the chat *
```

Vanliga meddelanden kan se ut så här:

```
[12:41] mats: hej  
[12:42] anna: tjena
```

Krav

- Tomt användarnamn ska hanteras.
- För långt användarnamn ska hanteras.
- Dubbla användarnamn ska inte krascha servern.
- Servern ska kunna meddela när någon ansluter och lämnar.

C-specifika frågor

- Hur stor buffert ska användarnamnet ha?
- Hur säkerställer du att strängen är nullterminerad?
- Hur tar du bort radbrytning från input?
- Hur undviker du buffer overflow?

17. Del 8: Klienten ska kunna skicka och ta emot samtidigt

En enkel klient som bara gör detta fungerar dåligt:

1. vänta på input från användaren
2. skicka input
3. vänta på svar från servern

Problemet är att meddelanden från andra användare kan komma när som helst.

Därför behöver klienten kunna:

- läsa tangentbordsinput
- samtidigt lyssna på inkommande data från servern

En rimlig lösning:

Huvudtråd:

läser från tangentbordet och skickar till servern

Mottagartråd:

läser från servern och skriver ut inkommande meddelanden

Krav

- Klienten ska visa inkommande meddelanden även när användaren inte skriver.
- Klienten ska kunna lämna med `/quit`.
- Klienten ska hantera om servern stängs.

Kontrollfrågor

- Vad händer med mottagartråden när socketen stängs?
- Behöver klienten använda mutex för utskrift?
- Hur undviker du att prompten blir rörig?

Första versionen behöver inte ha perfekt terminalgränssnitt. Funktion går före utseende.

18. Del 9: Hantera fel och avslut

I C måste du vara noggrann med felhantering.

Följande ska hanteras rimligt:

- `socket()` misslyckas
- `bind()` misslyckas
- `listen()` misslyckas
- `accept()` misslyckas
- `connect()` misslyckas
- `recv()` returnerar 0
- `recv()` returnerar negativt värde
- `send()` misslyckas
- klienten skickar `/quit`
- användaren trycker `Ctrl+C`

Du behöver inte lösa allt perfekt, men programmet ska inte krascha okontrollerat vid vanliga fel.

Saker att använda

Undersök:

- `perror()`
 - `errno`
 - `close()`
 - signalhantering med `signal()` eller `sigaction()`
-

19. Del 10: Skapa ett `chat`-kommando

När klienten fungerar kan du skapa ett enkelt kommando så att användaren slipper gå in i projektmappen.

Målet:

```
chat
```

ska starta klienten.

Du ska själv ta reda på:

- var egna kommandon kan placeras på Linux
- hur man gör en fil körbar
- hur ett litet shellsript kan starta din kompilerade klient
- hur scriptet hittar rätt binärfil

Kontrollfrågor

- Vad gör `$PATH`?
- Vad är `/usr/local/bin`?
- Varför bör du använda absoluta sökvägar i script?

20. Del 11: Kör servern stabilt med systemd

Under utveckling kan servern startas manuellt.

För mer permanent drift bör den köras som en tjänst.

Du ska skapa en `systemd`-service som:

- startar serverprogrammet
- körs som en vanlig användare
- startar om vid krasch
- kan aktiveras vid boot

Kommandon du sannolikt behöver

```
systemctl status namn.service
sudo systemctl start namn.service
sudo systemctl stop namn.service
sudo systemctl restart namn.service
sudo systemctl enable namn.service
journalctl -u namn.service -f
```

Du ska själv skriva servicefilen.

Kontrollfrågor

- Vad betyder `ExecStart`?
 - Vad betyder `WorkingDirectory`?
 - Varför bör tjänsten inte köras som root?
 - Vad händer om programmet skriver till relativ sökväg?
 - Hur ser du programmets `printf`-utskrifter när det körs som tjänst?
-

21. Del 12: SSH som åtkomstväg

Den enklaste lösningen är:

```
ssh användare@server-ip  
chat
```

Det räcker för grundversionen.

En mer avancerad lösning är att skapa en särskild Linuxanvändare som automatiskt hamnar i chatten vid SSH-inloggning.

Målbild:

```
ssh chatuser@server-ip
```

öppnar chatten direkt.

Detta kan göras med SSH-konfiguration, exempelvis genom att tvinga användaren att köra ett specifikt kommando vid inloggning.

Viktigt

Testa aldrig ändringar i SSH-konfiguration utan att ha en extra fungerande SSH-session öppen. Annars kan du låsa ute dig själv.

Frågor att undersöka

- Vad gör `ForceCommand`?
 - Vad innebär det att en användare får ett vanligt shell?
 - Hur kan man begränsa vad en SSH-användare får göra?
 - Hur kan SSH-nycklar användas istället för lösenord?
-

22. Säkerhet och nätverk

Rekommenderad design:

SSH-porten är öppen mot nätverket.
Chattservern är endast öppen lokalt på servern.

Det innebär att servern bör lyssna på:

127.0.0.1

och inte:

0.0.0.0

Kontrollfrågor

1. Vad är skillnaden mellan 127.0.0.1 och 0.0.0.0?
2. Varför är det säkrare att bara lyssna lokalt?
3. Vilken port måste vara öppen i brandväggen?
4. Behöver chattporten öppnas i UFW?
5. Vad händer om någon utifrån försöker nå chattporten direkt?

23. Testplan

Testa systemet stegvis.

Test 1: Kompilering

Förväntat resultat:

- `make` bygger både server och klient
- inga varningar från `gcc` som du inte förstår
- `make clean` tar bort byggfiler

Test 2: Servern startar

Förväntat resultat:

- servern startar utan fel
- den visar vilken host och port den lyssnar på

Test 3: En klient ansluter

Förväntat resultat:

- klienten ansluter

- servern visar att en klient anslutit
 - klienten kan skicka text
-

Test 4: Två klienter kommunicerar

Förväntat resultat:

- klient A skriver ett meddelande
 - klient B ser meddelandet
 - klient B svarar
 - klient A ser svaret
-

Test 5: En klient lämnar

Förväntat resultat:

- klienten skriver `/quit`
 - klientens socket stängs
 - servern kraschar inte
 - andra klienter är kvar
-

Test 6: Öväntat avbrott

Testa att stänga en klient med `Ctrl+C`.

Förväntat resultat:

- servern märker att klienten försvunnit
 - klienten tas bort ur klientlistan
 - övriga klienter fungerar fortfarande
-

Test 7: Server via systemd

Förväntat resultat:

- servern kan startas med `systemctl`
 - status visar att den kör
 - loggar syns via `journalctl`
-

Test 8: Anslutning via SSH

Förväntat resultat:

- användare kan logga in via SSH

- användare kan köra `chat`
 - flera SSH-sessioner kan delta samtidigt
-

24. Felsökningsverktyg

Använd följande verktyg under labben:

Kontrollera portar

```
ss -tulpn
```

eller:

```
lsof -i
```

Testa rå TCP-anslutning

```
nc 127.0.0.1 PORT
```

Felsök minnesfel

```
valgrind ./server
```

eller:

```
valgrind ./client
```

Kompilera med debug-symboler

Använd `-g` och felsök med:

```
gdb ./server
```

Läs systemd-loggar

```
journalctl -u namn.service -f
```

25. Vanliga C-fallgropar

Var särskilt uppmärksam på:

- buffertar som är för små
- strängar som saknar `\0`
- att `recv()` inte automatiskt skapar en C-sträng
- att `send()` inte nödvändigtvis skickar allt på en gång
- race conditions i klientlistan
- att glömma `close()` på sockets
- att skicka pekare till lokala variabler in i trådar
- att flera trådar skriver till terminalen samtidigt
- att inte kontrollera returvärden
- att anta att varje `recv()` motsvarar exakt ett meddelande

Särskilt viktig punkt:

TCP är en byte-ström, inte ett meddelandesystem.

Det betyder att ett meddelande kan komma uppdelat, eller att flera små meddelanden kan komma tillsammans. För en enkel labb kan du anta radbaserade meddelanden, men du ska känna till begränsningen.

26. Dokumentation som ska lämnas in

Skriv en kort README eller labbrapport med följande delar:

1. Syfte

2. Vad skulle programmet göra?

3. Systemdesign

4. Hur är server och klient uppdelade?

5. Hur går kommunikationen till?

6. Protokoll

7. Vad skickas först?

8. Hur ser vanliga meddelanden ut?

9. Hur avslutas anslutningen?

10. Implementation i C

11. Vilka sockets-funktioner använde du?

12. Hur hanterade du flera klienter?

13. Hur skyddade du delad data?

14. **Kompilering**

15. Hur bygger man programmen?

16. Vilka flaggor används?

17. **Testning**

18. Vilka tester gjorde du?

19. Vilka fel hittade du?

20. Hur löste du dem?

21. **Säkerhet**

22. Varför används SSH?

23. Varför lyssnar chattservern bara lokalt?

24. **Vidareutveckling**

25. Vad skulle du förbättra?

27. **Bedömningskriterier**

En godkänd lösning bör uppfylla:

- server och klient är skrivna i C
- programmen går att kompilera med `gcc`
- minst två klienter kan vara anslutna samtidigt
- meddelanden skickas mellan klienterna
- `/quit` fungerar
- servern kraschar inte när en klient lämnar
- chattprogrammet kan användas via SSH
- lösningen är dokumenterad

En starkare lösning kan dessutom ha:

- Makefile
- `pthread` med korrekt mutexhantering
- tydliga användarnamn
- tidsstämplar
- `/who` -kommando
- bättre felhantering

- Valgrind utan tydliga minnesfel
 - systemd-tjänst
 - särskild SSH-användare som hamnar direkt i chatten
-

28. Reflektionsfrågor

Besvara gärna efter labben:

1. Vad var svårast med sockets i C?
 2. Varför är `recv()` svårare att använda än vanlig tangentbordsinput?
 3. Varför behövs trådar eller annan samtidighet?
 4. Vilken data behövde skyddas med mutex?
 5. Vad händer om en klient kopplas bort mitt under en broadcast?
 6. Varför är TCP inte samma sak som separata meddelanden?
 7. Hur skulle du bygga stöd för flera rum?
 8. Hur skulle du spara meddelanden i en fil eller SQLite?
 9. Hur skulle systemet behöva ändras för att bli publikt?
 10. Vad tillför SSH i denna arkitektur?
-

29. Möjliga extrauppgifter

Extra 1: `/who`

Lägg till ett kommando som visar vilka användare som är anslutna.

Extra 2: Chathistorik i minnet

Spara de senaste 20 meddelandena i en array och visa dem när en ny klient ansluter.

Fundera på:

- hur arrayen ska skyddas
 - vad som händer när den blir full
 - om historiken ska innehålla systemmeddelanden
-

Extra 3: Loggfil

Spara alla meddelanden i en loggfil.

Fundera på:

- filrättigheter
- radformat
- om flera trådar kan skriva samtidigt
- om filen kan växa obegränsat

Extra 4: SQLite

Spara meddelanden i SQLite.

Fundera på tabeller som:

```
users
messages
rooms
```

Detta kräver att du länkar mot SQLite-biblioteket.

Extra 5: Flera rum

Lägg till kommandon som:

```
/join general
/join linux
/join random
```

Fundera på hur servern ska hålla reda på vilket rum varje klient tillhör.

Extra 6: Bättre terminalgränssnitt

Undersök `ncurses` för att skapa ett mer ordnat terminalgränssnitt med separat inputrad och meddelandefönster.

Extra 7: Direkt SSH-chatt

Skapa en särskild SSH-användare som automatiskt startar klienten vid inloggning.

Mål:

```
ssh chatuser@server-ip
```

ska öppna chatten direkt.

30. Slutsats

I denna labb ska du bygga ett enkelt terminalbaserat chattforum i C. Fokus ligger på att förstå hur sockets, trådar och delad data fungerar i praktiken.

Labben är medvetet öppen. Du får en tydlig målbild, men behöver själv fatta designbeslut och skriva implementationen. Det gör uppgiften mer lik ett riktigt C-projekt än en kopiera-och-kör-labb.